Федеральное государственное автономное образовательное учреждение
высшего образования
Национальный исследовательский университет ИТМО
Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Тестирование программного обеспечения»

Отчёт

по лабораторной работе №1

Вариант: 7023

Студент:
Чусовлянов Максим Сергеевич
Группа P3307

Преподаватель:
Бострикова Дарья Константиновна

г. Санкт-Петербург, 2026 г.

Описание задания

Лабораторная работа #1

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

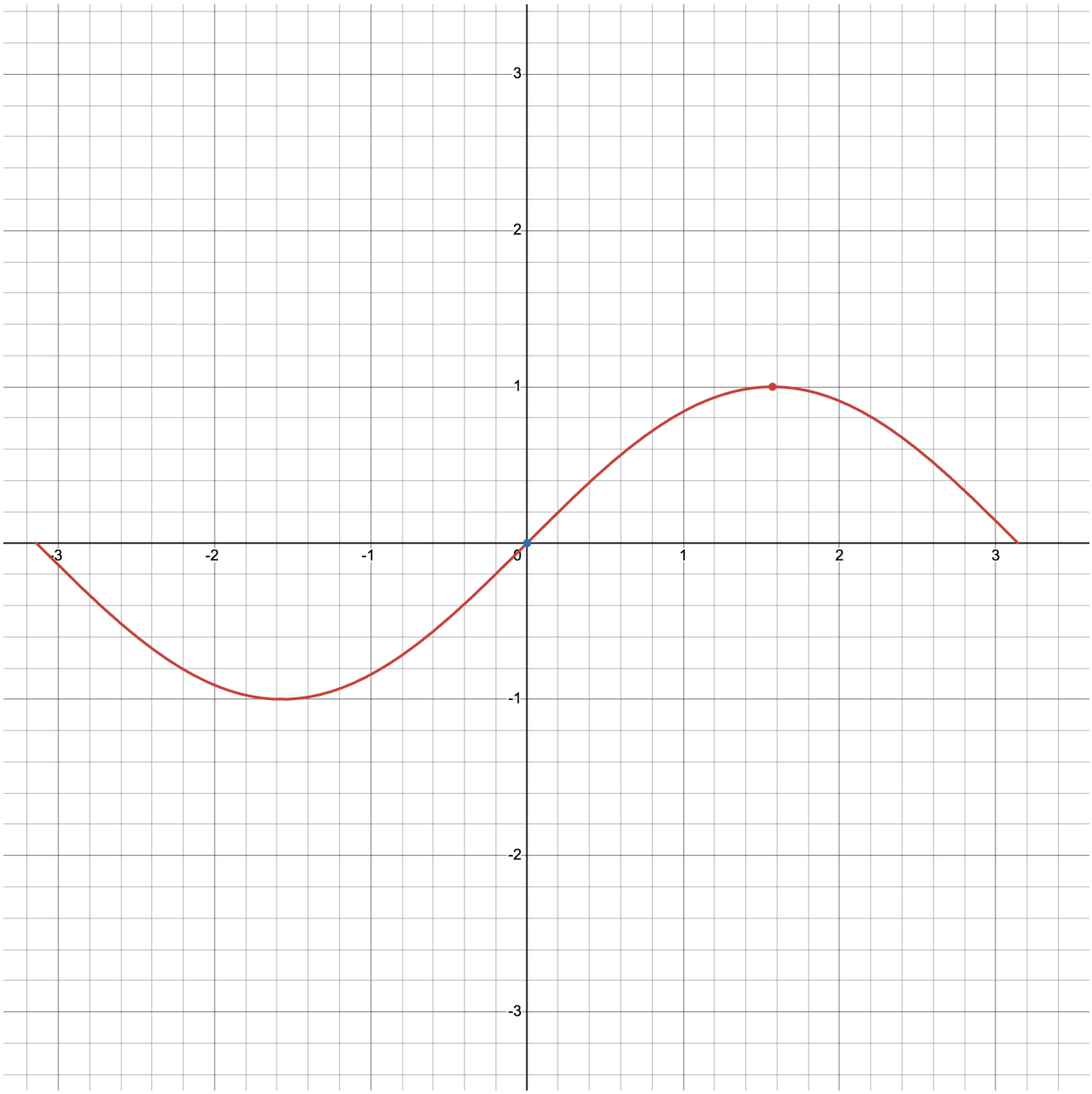
Введите вариант: 7023

1. Функция sin(x)
2. Программный модуль для работы с B+ деревьями (максимальное количество элементов в ключе - 7, http://www.cs.usfca.edu/~galles/visualization/BPlusTree.html)
3. Описание предметной области:

> И все, как это обычно бывает с теориями, было мило и славно, пока Виит Вужагиг не заявил вдруг, что он нашел эту планету, побывал на ней, и работал там шофером лимузина у семьи дешевых зеленых авторучек на пружинках. После этого за ним приехали, поместили его в запертую комнату, где он написал книгу, и, наконец, отправили туда, где люди не платят налогов -- обычная участь тех, кто решил публично повалять дурака.

Выполнение

Задание 1: $\sin(x)$, разложение в степенной ряд



Выбор тестовых точек

Выберем характерные точки для тестирования:

- $x = 0$ — тривиальная точка, $\sin(0) = 0$;
- $x = \frac{\pi}{2}$ — максимум функции, $\sin(\frac{\pi}{2}) = 1$;
- $x = \pi$ — переход через ноль, $\sin(\pi) = 0$;
- $x = -\frac{\pi}{2}$ — минимум, $\sin(-\frac{\pi}{2}) = -1$;
- $x = 2\pi$ — период, $\sin(2\pi) = 0$;
- $x = 3\frac{\pi}{2}$ — второй минимум, $\sin(3\frac{\pi}{2}) = -1$;
- $\pm$NaN, $\pm\infty$ — специальные значения IEEE 754, должны возвращать NaN;
- Большие аргументы ($x = 10, 100, -50$) — проверка нормализации;
- Случайные значения — сравнение с Math.sin() из стандартной библиотеки.

Таблица тестовых данных

x	$\sin(x)$ (ожидаемое)	Класс точки
0.0	0.0	нуль
$\frac{\pi}{6} \approx 0.5236$	0.5	табличное значение
$\frac{\pi}{4} \approx 0.7854$	$\frac{\sqrt{2}}{2} \approx 0.7071$	табличное значение
$\frac{\pi}{3} \approx 1.0472$	$\frac{\sqrt{3}}{2} \approx 0.8660$	табличное значение
$\frac{\pi}{2} \approx 1.5708$	1.0	максимум
$\pi \approx 3.1416$	0.0	ноль
$3\frac{\pi}{2} \approx 4.7124$	-1.0	минимум
$2\pi \approx 6.2832$	0.0	период
$-\frac{\pi}{2}$	-1.0	минимум
10.0	-0.5440 (Math.sin)	большой аргумент
100.0	-0.5064 (Math.sin)	большой аргумент
NaN	NaN	специальное значение
$+\infty$	NaN	специальное значение
$-\infty$	NaN	специальное значение

Стратегия тестового покрытия

Реализованы следующие группы тестов:

1. **Граничные и табличные значения** — 11 конкретных точек через `@CsvSource`: охватывают нули, максимумы, минимумы, π-кратные значения.
2. **Специальные значения** — NaN, $\pm\infty$: функция должна вернуть NaN, не выбрасывая исключения.
3. **Математические свойства**:
   - Нечётность: $\sin(-x) = -\sin(x)$ — 7 точек через `@ValueSource`;
   - Периодичность: $\sin(x) = \sin(x \pm 2\pi)$ — 6 точек.
4. **Большие аргументы** — $x \in \{10, 100, -50, 37.5\}$: проверка нормализации.
5. **Сходимость ряда** — сравнение погрешности при $n = 5$ и $n = 15$ членах: погрешность должна убывать.

6. **Некорректный параметр** — `terms <= 0` должен бросать `IllegalArgumentException`.
7. **Случайное тестирование (фаззинг)** — 10 000 случайных значений $x \in [-20\pi, 20\pi]$, сравнение с `Math.sin()` с допуском $10^{\{-5\}}$.

Задание 2: B+ дерево

Описание алгоритма

B+ дерево — самобалансирующееся дерево поиска, в котором все данные хранятся в листовых узлах; внутренние узлы содержат только ключи-разделители. Листья связаны в односвязный список, что обеспечивает эффективный диапазонный поиск.

Параметры реализации:

- `MAX_KEYS = 7` — максимум ключей в узле;
- переполнение наступает при `keyCount = 8`;
- при расщеплении листа: оба потомка получают по 4 ключа;
- при расщеплении внутреннего узла: средний ключ (индекс 3) уходит наверх, левый потомок получает 3 ключа, правый — 4.

Характерные точки алгоритма INSERT

Точка	Описание
A	Вызов операции `insert`
B	Дерево пусто — создаётся листовой корень
C	Найден лист для вставки
D	В листе есть место — вставка без расщепления
E	Лист переполнен — выполняется расщепление
F	Расщеплённый лист был корнем — создаётся новый корень
G	Разделитель вставлен в родительский узел без его переполнения
H	Родительский узел переполнен — выполняется расщепление внутреннего узла
I	Расщеплённый внутренний узел был корнем — создаётся новый корень

Характерные точки алгоритма SEARCH

Точка	Описание
J	Вызов операции `search`
K	Ключ найден в листовом узле
L	Ключ не найден (или дерево пусто)

Наборы данных и эталонные последовательности

№	Описание набора данных	Эталонная последовательность
1	Вставка в пустое дерево: `insert(42)`	`A → B`

№	Описание набора данных	Эталонная последовательность
2	Вставка без расщепления: `insert(1); insert(2)`	`A → C → D`
3	Заполнение листа до предела: `insert(1..6); insert(7)`	`A → C → D`
4	Переполнение листового корня: `insert(1..7); insert(8)`	`A → C → E → F`
5	Расщепление листа при наличии родителя с местом: `insert(1..11); insert(12)`	`A → C → E → G`
6	Расщепление листа при полном корне: `insert(1..35); insert(36)`	`A → C → E → H → I`
7	Поиск в пустом дереве: `search(1)`	`J → L`
8	Поиск существующего ключа: `insert(5); search(5)`	`J → K`
9	Поиск отсутствующего ключа: `insert(1,2,3); search(99)`	`J → L`
10	Поиск в многоуровневом дереве: `insert(1..16); search(13)`	`J → K`

Дополнительные проверки

Помимо трассировки характерных точек, тесты проверяют:

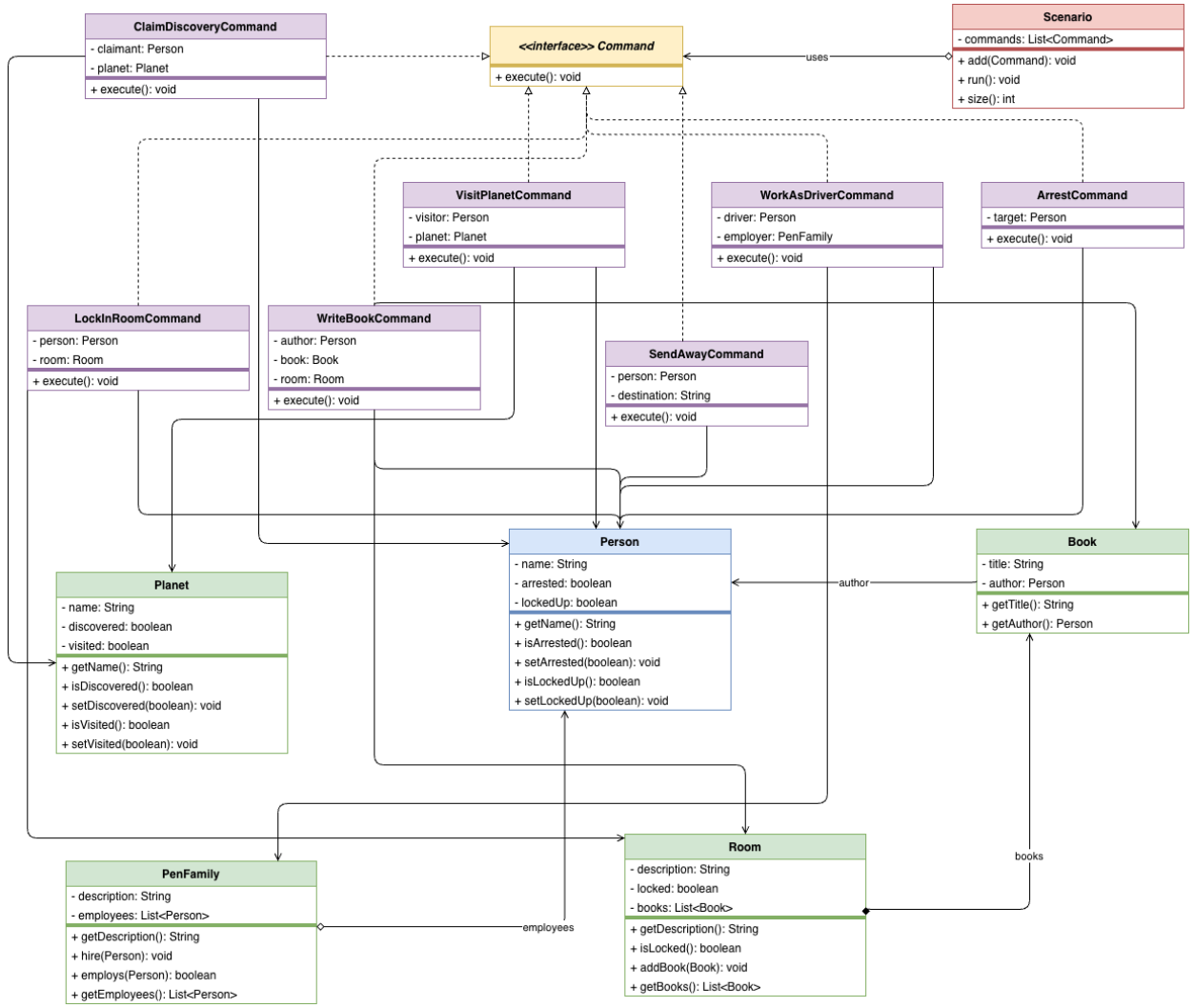
- корректность структуры дерева (все ключи в отсортированном порядке через `getAllKeys()`);
- высоту дерева (`getHeight()`) после серий вставок;
- независимость результата от порядка вставки;
- корректность удаления и диапазонного поиска.

Задание 3: Доменная модель

Исходный текст предметной области

«И все, как это обычно бывает с теориями, было мило и славно, пока **Виит Вужагиг** не заявил вдруг, что он **нашел** эту **планету**, **побывал** на ней, и **работал** там **шофёром лимузина** у **семьи дешёвых зелёных авторучек на пружинках**. После этого за ним **приехали**, **поместили** его в **запертую комнату**, где он **написал книгу**, и, наконец, **отправили** туда, где люди не платят налогов — обычная участь тех, кто решил публично повалять дурака.»

Сущности доменной модели



Из текста выделены следующие объекты предметной области:

Класс	**Поля**	**Описание**
`Person`	`name`, `arrested`, `lockedUp`	Виит Вужагиг; хранит состояния «арестован» и «изолирован»
`Planet`	`name`, `discovered`, `visited`	Планета; фиксирует, открыта ли она и посещалась ли
`PenFamily`	`description`, `employees`	Семья авторучек-работодатель; список сотрудников
`Room`	`description`, `locked`, `books`	Запертая комната; список написанных книг
`Book`	`title`, `author`	Книга с автором

Действия — паттерн Command

Каждое действие из текста реализовано как отдельная команда, реализующая интерфейс `Command`:

Команда	**Действие**
`ClaimDiscoveryCommand`	Объявление об открытии планеты → `planet.discovered = true`
`VisitPlanetCommand`	Посещение планеты → `planet.visited = true`
`WorkAsDriverCommand`	Найм шофёром → `penFamily.employees.add(person)`
`ArrestCommand`	Арест → `person.arrested = true`
`LockInRoomCommand`	Заключение в комнату → `person.lockedUp = true` (исключение, если комната не заперта)
`WriteBookCommand`	Написание книги → `room.books.add(book)` (исключение, если автор не изолирован)
`SendAwayCommand`	Отправка в место назначения → вывод сообщения

Класс `Scenario` хранит список команд и выполняет их последовательно методом `run()`.

Ссылка на код

https://github.com/Vaneshik/tpo-lab1

Выводы

В ходе данной лабораторной работы я познакомился с библиотекой для тестирования JUnit и написал собственные классы и методы для тестирования некоторых алгоритмов, а также доменной модели.